Day 75: Scaled Dot-Product Attention — What the 1/√d Actually Does

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 6 | Complete study guide

Watch: Why Scaling by the Square Root of Dimensions Matters in Attention (Learn With Jay) | Self-Attention Using Scaled Dot-Product Approach (Machine Learning Studio)

Script: `day75_scaled_dot_product_attention.py` · every number in this guide is copied by program from `day75_run_output.txt`, the real output of that script.

The Transformer divides every attention score by √d. Today follows the reason step by step. With inputs of spread 1, q · k has spread √d (15.986 at d = 256). That makes the softmax nearly one-hot, and the gradient through it nearly vanishes. On a recall task that needs attention, the scaled model is essentially perfect within 250 steps at d = 64 and 128 (0.999 and 1.000); the unscaled one is still near 0.128. One more experiment sharpens the lesson: unscaled scores with smaller initial query and key weights train exactly as well. In this setting the 1/√d matters because of **where training starts**.

0. Where today fits

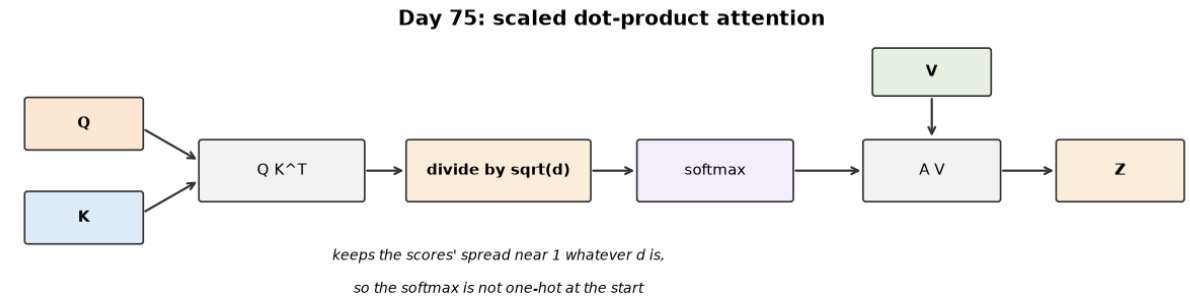
DEFINITION — start here, no prior knowledge assumed

Scaled dot-product attention (Vaswani et al., 2017) is Day 74's attention with one change: the scores are divided by √d, where d is the length of the query and key vectors. $A = \text{softmax}(Q K^T / \sqrt{d})$, $Z = A V$. It is the core operation of every Transformer.

REAL-WORLD SCENARIO

Grading on a curve: if exams get longer (more questions, larger d), raw totals spread out more, and a fixed "top mark wins everything" rule becomes winner-take-all. Dividing by the expected spread keeps the comparison meaningful whatever the exam length.

SYSTEM DIAGRAM



Scaled dot-product attention: scores, divided by √d, softmax, weighted sum.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. How large is q · k?

DEFINITION — start here, no prior knowledge assumed

A dot product of two d-long vectors is a sum of d products. If each number has mean 0 and spread 1, each product has spread 1, and the sum of d independent ones has spread √d.

REAL-WORLD SCENARIO

Adding up d coin tosses scored +1 or –1: the total typically lands about √d away from zero.

MATHEMATICS

$$q \cdot k = \Sigma_{i=1..d}\, q_i\, k_i; \quad Var(q_i\, k_i) = 1 \;\Rightarrow\; Var(q \cdot k) = d, \quad spread\ \sqrt{d}$$

$$dividing\ by\ \sqrt{d}: \quad spread\ of\ (q \cdot k)/\sqrt{d} = 1\ for\ every\ d$$

SPECIFICATION

Data	10,000 random pairs of vectors with numbers of mean 0 and spread 1, for d = 4 to 1,024.

Why this test

It checks the premise of the whole argument with real numbers.

IMPLEMENTATION

```
for d in [4, 16, 64, 256, 1024]:
    q, k = rng.randn(10000, d), rng.randn(10000, d)
    s = (q * k).sum(axis=1)
    print(f"{d:>6}{s.std():>17.3f}{np.sqrt(d):>9.3f}{(s / np.sqrt(d)).std():>27.3f}")
```

ACTUAL OUTPUT

```
     d  spread of q . k  sqrt(d)  spread of q . k / sqrt(d)
     4            2.009    2.000                      1.005
    16            4.004    4.000                      1.001
    64            7.982    8.000                      0.998
   256           15.986   16.000                      0.999
  1024           32.012   32.000                      1.000
Actual output from this lesson's run.
```

What it means

The spread grows exactly as √d (2.009 at d = 4, 32.012 at d = 1,024), and dividing by √d brings it back to 1 at every size (1.000 at 1,024).

WHY CHOOSE THIS?

One division keeps the scores on the same scale for every head size.

WHY NOT THIS? (TRADEOFFS)

It assumes inputs of spread 1; with other scales (Day 74's small embeddings) the scores are small anyway.

2. What large scores do to the softmax

DEFINITION — start here, no prior knowledge assumed

The softmax turns scores into weights. When one score is much larger than the others, nearly all the weight goes to it: the softmax becomes **one-hot**. **Entropy** measures the spread: log 16 = 2.773 for an even spread over 16 keys, 0 for one-hot.

REAL-WORLD SCENARIO

A vote where one candidate's tally is far ahead: the result is decided, and small changes no longer move it.

SPECIFICATION

Data	5,000 random queries against 16 random keys each, numbers of spread 1.

Why this test

It shows the step from “large scores” to “one-hot attention”.

IMPLEMENTATION

```
for A in [softmax(S), softmax(S / np.sqrt(d))]:
    r += [A.max(axis=1).mean(), (-(A * np.log(A + 1e-30)).sum(axis=1)).mean()]
rows[d] = r
```

ACTUAL OUTPUT

```
     d  unscaled: max weight  entropy   scaled: max weight  entropy
     4                0.421    1.780                0.232    2.387
    16                0.682    0.911                0.241    2.369
    64                0.843    0.414                0.244    2.364
   256                0.922    0.193                0.246    2.359
  1024                0.959    0.098                0.246    2.358
Actual output from this lesson's run.
```

What it means

Unscaled, the largest weight climbs from 0.421 to 0.959 and the entropy falls to 0.098: at d = 1,024 each query looks at essentially one key, chosen by chance. Scaled, the weights stay spread at every size (entropy about 2.36).

WHY CHOOSE THIS?

A spread-out start lets every key receive some weight, so learning can move it.

WHY NOT THIS? (TRADEOFFS)

A model that needs sharp attention must learn to sharpen it.

3. What it does to the gradient

DEFINITION — start here, no prior knowledge assumed

The gradient of a softmax is A × (dA – Σ A dA): where one weight is 1 and the rest are 0, it is almost zero. A softmax that is one-hot is **frozen**: small changes to the scores change nothing, so nothing reaches the queries and keys.

REAL-WORLD SCENARIO

Pushing on a door that is already fully open: effort no longer moves it.

MATHEMATICS

$$dS = A \times (dA - \Sigma_j A_j \, dA_j); \quad \text{one-hot } A \Rightarrow dS \approx 0$$

$$\text{scaled: } S = (q \cdot k) / \sqrt{d}, \text{ so the gradient reaching } q \cdot k \text{ is } dS / \sqrt{d}$$

SPECIFICATION

Data	Section 2's queries and keys, and a random gradient of spread 1 arriving at the weights.
Measured	Median size of the gradient reaching q · k, and the share of queries where it is below 1e-6.

Why this test

It is the last step of the argument: one-hot weights ⇒ no gradient.

IMPLEMENTATION

```
for c in [1.0, 1 / np.sqrt(d)]:
    dS = softmax_backward(dA, softmax(c * S)) * c
    size = np.linalg.norm(dS, axis=1)
    r += [np.median(size), (size < 1e-6).mean()]
```

ACTUAL OUTPUT

```
     d  unscaled: gradient size   frozen  scaled: gradient size   frozen
     4                 2.68e-01    0.000               1.31e-01    0.000
    16                 1.95e-01    0.000               6.71e-02    0.000
    64                 5.82e-02    0.015               3.33e-02    0.000
   256                 3.45e-03    0.162               1.69e-02    0.000
  1024                 1.10e-05    0.438               8.38e-03    0.000
(median over the 5,000 queries)
Actual output from this lesson's run.
```

What it means

The effect is not what one might first guess. At small and medium sizes the **unscaled** gradient is larger (at d = 64, 5.82e-02 against 3.33e-02), because the scaling itself multiplies the gradient by 1/√d. The difference is at large sizes: unscaled, the gradient collapses (1.10e-05 at d = 1,024) and 0.438 of the queries are frozen; scaled, none are (0.000), and the gradient shrinks only smoothly. The scaling's job is to stop the softmax freezing, not to make gradients bigger.

WHY CHOOSE THIS?

No query freezes, at any size.

WHY NOT THIS? (TRADEOFFS)

Gradients are 1/√d smaller at small sizes, where nothing would have frozen.

4. Training: associative recall

DEFINITION — start here, no prior knowledge assumed

A task only attention can solve: each example shows 16 (key, value) pairs and a query key; the answer is the value stored with that key. The model reads the pairs with one scaled (or unscaled) dot-product attention read from the query, then guesses one of 32 values.

REAL-WORLD SCENARIO

A coat check with 16 tickets: hand over a ticket number, get back the right coat. You must find the matching ticket; no amount of guessing from the others helps.

MATHEMATICS

$$x_j = E_k[key_j] + E_v[value_j]; \quad x_q = E_k[query] + e_q$$

$$a = softmax(\, c\,(x_q W_q) \cdot (x_j W_k)\,), \quad c = 1/\sqrt{d} \text{ or } 1; \quad answer = softmax(\, (\Sigma_j\, a_j\, x_j W_v)\, W_o\,)$$

SPECIFICATION

Task	16 pairs with different keys out of 32, random values out of 32; guessing gives 1/32 = 0.031.
Training	Embeddings of spread 1 (as after a Transformer's normalisation); plain SGD lr 0.1, clip 1.0, batches of 64, 2,000 steps; d = 16, 64, 128; three seeds; 2,000 fixed test examples.

Why this test

Sections 1–3 are about one random step. This asks whether the effect matters for learning.

IMPLEMENTATION

```
def forward(self, keys, vals, qkey):
    P = self.P
    X = P["Ek"][keys] + P["Ev"][vals]
    xq = P["Ek"][qkey] + P["Eq"]
    q, K, Vv = xq @ P["Wq"], X @ P["Wk"], X @ P["Wv"]
    S = self.c * np.einsum("md,mnd->mn", q, K)
    A = softmax(S)
    ctx = np.einsum("mn,mnd->md", A, Vv)
    p = softmax(ctx @ P["Wo"] + P["bo"])
    self.cache = dict(keys=keys, vals=vals, qkey=qkey, X=X, xq=xq, q=q, K=K, V=Vv, S=S, A=A,
        ctx=ctx, p=p)
    return p
```

IMPLEMENTATION

```
dS = softmax_backward(dA, C["A"]) * self.c
```

The gradient check of this model, scaled and unscaled (size 4, every entry, Day 62's pass rule):

ACTUAL OUTPUT

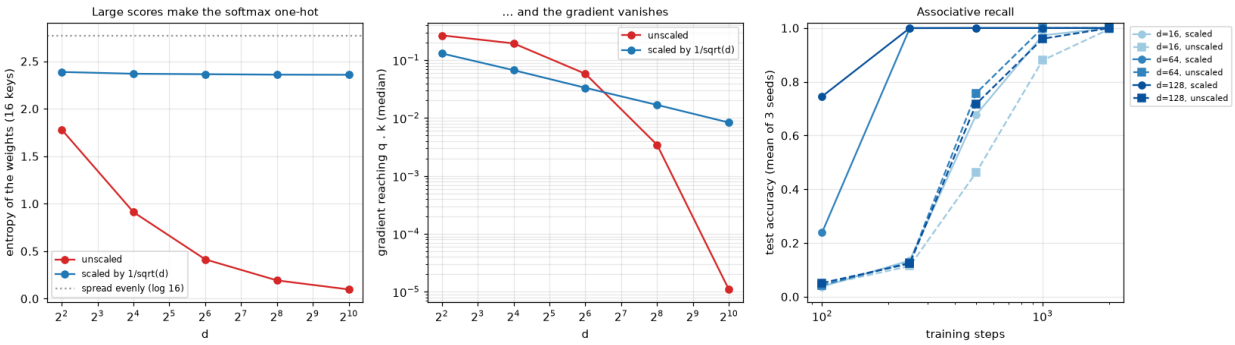
```
scale none : largest |numeric - analytic| = 1.14e-09; every entry passes: True
scale sqrt : largest |numeric - analytic| = 4.44e-10; every entry passes: True
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
    d  scaling  attention entropy at start    @100    @250    @500   @1000   @2000
   16     none                        0.548   0.043   0.114   0.462   0.880   0.995
   16     sqrt                        1.931   0.037   0.135   0.677   0.973   1.000
   64     none                        0.235   0.043   0.128   0.758   1.000   1.000
   64     sqrt                        1.843   0.239   0.999   1.000   1.000   1.000
  128     none                        0.142   0.051   0.124   0.717   0.960   1.000
  128     sqrt                        1.741   0.745   1.000   1.000   1.000   1.000
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
per seed, accuracy at 250 steps:
  d = 16  none: 0.199 0.109 0.035   sqrt: 0.210 0.145 0.051
  d = 64  none: 0.209 0.103 0.070   sqrt: 1.000 1.000 0.999
  d = 128 none: 0.140 0.119 0.112   sqrt: 1.000 1.000 1.000
Actual output from this lesson's run.
```



Left: entropy of random attention by size. Middle: the gradient reaching q · k. Right: recall accuracy during training, solid scaled, dashed unscaled.

What it means

At d = 64 and 128 the scaling makes training several times faster. Scaled, every seed is at 1.000 or 0.999 after 250 steps; unscaled, the seeds are between 0.070 and 0.209, and catch up only by 1,000–2,000 steps. The untrained unscaled models start nearly one-hot (entropy 0.235 and 0.142, against log 16 = 2.77), looking at one random pair. At d = 16 the two are close (0.677 against 0.462 at 500 steps): the scores are not large enough there to freeze anything.

WHY CHOOSE THIS?

Learns the task in a fraction of the steps at realistic head sizes.

WHY NOT THIS? (TRADEOFFS)

Both versions get there eventually here; the gain is speed, on this task and optimiser.

5. Why √d?

DEFINITION — start here, no prior knowledge assumed

The same model with three factors in front of q · k: 1 (no scaling), 1/√d and 1/d.

REAL-WORLD SCENARIO

Grading on a curve, too much or too little: dividing by the class size instead of the spread flattens every score to a tie.

SPECIFICATION

Runs	d = 64, three seeds each, everything else as Section 4.

Why this test

If all that mattered were “smaller scores”, 1/d would work even better.

IMPLEMENTATION

```
self.c = {"none": 1.0, "sqrt": 1 / np.sqrt(d), "d": 1 / d}[scale]
```

ACTUAL OUTPUT

```
      factor  entropy at start    @100    @250    @500   @1000   @2000
           1             0.235   0.043   0.128   0.758   1.000   1.000
  1/sqrt(64)             1.843   0.239   0.999   1.000   1.000   1.000
        1/64             2.751   0.076   0.209   0.967   0.999   1.000
Actual output from this lesson's run.
```

What it means

1/d overshoots: the attention starts almost perfectly even (entropy 2.751), every pair looks the same, and learning to tell them apart is slow (0.209 at 250 steps). 1/√d starts spread but not flat (entropy 1.843) and learns fastest. Neither too sharp nor too flat: that is what keeping the spread at 1 achieves.

WHY CHOOSE THIS?

1/√d is the factor that keeps the spread at 1, and it learned fastest here.

WHY NOT THIS? (TRADEOFFS)

One size, one task.

6. Scaling, or just a smaller start?

DEFINITION — start here, no prior knowledge assumed

Unscaled scores, but the query and key weights start $64^{1/4} \approx 2.83$ times smaller. Each score is a product of a query and a key, so it starts $8 = \sqrt{64}$ times smaller: exactly the scaled model's starting scores.

REAL-WORLD SCENARIO

Two cars set at the same starting line: do they finish together, or does one have a better engine?

MATHEMATICS

$$(x\,W_q / 64^{1/4}) \cdot (x\,W_k / 64^{1/4}) = (x\,W_q \cdot x\,W_k) / 8 \quad = \textit{the scaled model's score at the start}$$

SPECIFICATION

Runs	d = 64, three seeds: scaled; unscaled with the smaller start; unscaled as in Section 4.

Why this test

Section 4 cannot tell whether the scaling helps during the whole of training or only by starting the model in a better place.

IMPLEMENTATION

```
shrink = 64 ** -0.25
a, b = Recall(64, "sqrt", seed=0), Recall(64, "none", seed=0, qk_init=shrink)
pa, pb = a.forward(*TEST[:3]), b.forward(*TEST[:3])
```

ACTUAL OUTPUT

```
largest difference between the two models' starting attention weights: 1.05e-15
model                                @100    @250    @500   @1000   @2000
scaled (1/sqrt(d))                  0.239   0.999   1.000   1.000   1.000
unscaled, smaller Wq and Wk         0.206   1.000   1.000   1.000   1.000
unscaled (Section 4)                0.043   0.128   0.758   1.000   1.000
Actual output from this lesson's run.
```

What it means

The two start identically (1.05e-15 difference) and train identically for practical purposes: 0.206 against 0.239 at 100 steps, both at 0.999 or above by 250. **Here, the whole benefit of the scaling comes from the starting point.** That is a real finding, not the textbook story of the scaling helping throughout training. It may not hold with other optimisers, such as Adam (which reacts to gradient size differently), or in deep models where the scores grow during training; neither was tested here.

WHY CHOOSE THIS?

Shows what the scaling actually does in this setting.

WHY NOT THIS? (TRADEOFFS)

One optimiser (plain SGD), one task, one size.

7. One larger size

DEFINITION — start here, no prior knowledge assumed

The Section 4 comparison at d = 256, one seed (a run takes several minutes here).

REAL-WORLD SCENARIO

The same coat check with much longer ticket numbers.

ACTUAL OUTPUT

```
  scaling    @100    @250    @500   @1000   @2000
     none   0.059   0.079   0.337   1.000   1.000
     sqrt   0.953   1.000   1.000   1.000   1.000
Actual output from this lesson's run.
```

What it means

The gap is at least as large as at d = 128: scaled 0.953 after 100 steps, unscaled 0.059 (still 0.337 at 500). One seed only, so no more than a check that the pattern continues.

WHY CHOOSE THIS?

Consistent with Section 4 at a larger size.

WHY NOT THIS? (TRADEOFFS)

A single seed.

Interview preparation — technical questions

Q: Write scaled dot-product attention.

A: Z = softmax(Q K^T / sqrt(d)) V, with masked positions set to minus infinity before the softmax.

Q: Why divide by sqrt(d)?

A: With numbers of spread 1, q . k has spread sqrt(d); dividing restores spread 1 so the softmax does not start one-hot.

Q: What happens to gradients without the scaling at large d?

A: The softmax freezes: here 44% of queries had a gradient below 1e-6 at d = 1,024.

Q: Why not divide by d?

A: It flattens the scores: entropy 2.751 of a possible 2.77, and slower learning (0.209 at 250 steps).

Q: Is the scaling equivalent to a smaller initialisation?

A: At the start, exactly; in today's SGD runs they also trained the same. Not tested with Adam or deeper models.

Interview preparation — theoretical questions

Q: Does the scaling make gradients larger?

A: No: at d = 64 the unscaled gradient was larger. It prevents freezing at large d.

Q: Why does a one-hot start learn slowly?

A: A likely reason (not measured here): each query reads one random key, the right key has almost zero weight, and so its score receives almost no gradient.

Q: When does the scaling matter little?

A: When scores are small anyway: small d (16 here), or small embeddings (Day 74).

Q: Additive attention has no such scaling. Why?

A: Its score passes through tanh and one vector v, so it is bounded by |v| and does not grow with d in the same way.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> With numbers of spread 1, q · k grows like √d, the softmax turns one-hot and its gradient freezes. Dividing by √d keeps the scores at spread 1. On a recall task that needs attention this made learning several times faster at d = 64 and 128, while 1/d overshot into flat attention. And in this setting the whole benefit came from the starting point: shrinking the initial query and key weights did exactly as well.

- Spread of q . k = sqrt(d): 2.009 at d = 4, 32.012 at d = 1,024.
- Unscaled softmax at d = 1,024: largest weight 0.959, entropy 0.098, 44% of queries frozen.
- Recall, 250 steps, d = 64 / 128: scaled 0.999 / 1.000, unscaled 0.128 / 0.124.
- 1/d: too flat, slow. 1/sqrt(d): fastest.
- Smaller initial Wq, Wk without scaling: the same training as scaled.
- The scaling prevents freezing; it does not make gradients bigger.

Suggested watch order, if starting from zero

- 1. Self-Attention Using Scaled Dot-Product Approach (Machine Learning Studio) — scaled dot-product self-attention, step by step.

- 2. Why Scaling by the Square Root of Dimensions Matters in Attention (Learn With Jay) — the variance argument for the √d.

Day 76 preview

- **Multi-head attention, the week's build.** Several scaled dot-product attentions run side by side on smaller projections, and their outputs are joined. Day 76 assembles it from this week's exercise scripts.
- Today's exercise script, `day75_exercise.py`, is scaled dot-product attention as one reusable, gradient-checked function with a mask: the piece each head will run.